

ANDROID NETWORKING

Retrofit & Volley

Core concepts, the basics you need first, and a practical comparison

Agenda

01

Why networking libraries

02

Basics: HTTP, JSON, threads & permissions

03

Volley: architecture & code

04

Retrofit: architecture & code

05

Head-to-head comparison

06

Picking the right tool

Why not just write it yourself?

Every Android app that talks to a server runs into the same four problems



Boilerplate

URLConnection needs manual streams, buffers and connection setup for every single call.



Threading

The UI thread cannot make network calls — every request needs its own background thread.



Parsing

Raw JSON strings have to be manually turned into Java or Kotlin objects.



Reliability

Retries, caching, cancellation and error handling are easy to get wrong by hand.

Raw networking, circa 2010–2013

Fetching the same `/students/102` endpoint — no library, just the Android SDK

• AsyncTask + HttpURLConnection

```
class FetchStudentTask(val callback: (String?, Exception?) -> Unit)
    : AsyncTask<String, Void, String>() {
    var lastException: Exception? = null
    // runs on a background thread AsyncTask spins up for you
    override fun doInBackground(vararg urls: String): String? {
        var connection: HttpURLConnection? = null
        try {
            val url = URL(urls[0])
            connection = url.openConnection() as HttpURLConnection
            connection.requestMethod = "GET"
            connection.connect()
            if (connection.responseCode != HttpURLConnection.HTTP_OK)
                throw IOException("HTTP ${connection.responseCode}")

            // manually read the raw byte stream into a string
            val reader = BufferedReader(InputStreamReader(connection.inputStream))
            val result = reader.readLine()
            reader.close()
            return result
        } catch (e: Exception) {
            lastException = e; return null
        } finally {
            connection?.disconnect() // forget this = leaked socket
        }
    }

    // AsyncTask's one convenience: lands back on the main thread
    override fun onPostExecute(result: String?) {
        if (result == null) { callback(null, lastException); return }
        try {
            JSONObject(result) // manual parsing, again
            callback(result, null)
        } catch (e: JSONException) { callback(null, e) }
    }
}

// usage — and this whole class only fetches ONE endpoint
FetchStudentTask { result, error ->
    if (error != null) showError(error) else showStudent(result)
}.execute("https://api.jipmer.edu/students/102")
```

No queue or priority

5 requests → 5 racing threads, no cap or ordering.

No caching

Every screen re-fetches — you'd hand-roll a disk cache.

Leak-prone

Activity ref held across rotation = classic leak.

One class per call

These ~40 lines repeat for every single endpoint.

Instant Auto-Complete Search Fields



Tag-Based Cancellation

Instantly aborts stale requests on keypress via `cancelAll()`.

Race Condition Prevention

Prevents older delayed responses from overwriting fresh screen data.



Low-Latency RPC

Minimal object instantiation overhead per keystroke query.

Fast-Scrolling Thumbnail Feeds

NetworkImageView Integration

Built-in bitmap cache memory manager for lists.

Automatic View Recycling

Cancels off-screen downloads during rapid scrolling.

Memory Protection

Prevents OutOfMemoryError crashes in heavy image catalogs.

Basics: HTTP & REST

Both libraries sit on top of **HTTP**. A REST API exposes resources as URLs, and a client acts on them with a method:

- **GET** — read a resource
- **POST** — create a resource
- **PUT** — replace a resource
- **DELETE** — remove a resource

A request carries headers (like **Content-Type**) and a body; the server replies with a status code and a body.

GET /api/students/102

Code	Meaning
200	OK
201	Created
400	Bad request
404	Not found
500	Server error

Basics: JSON & parsing

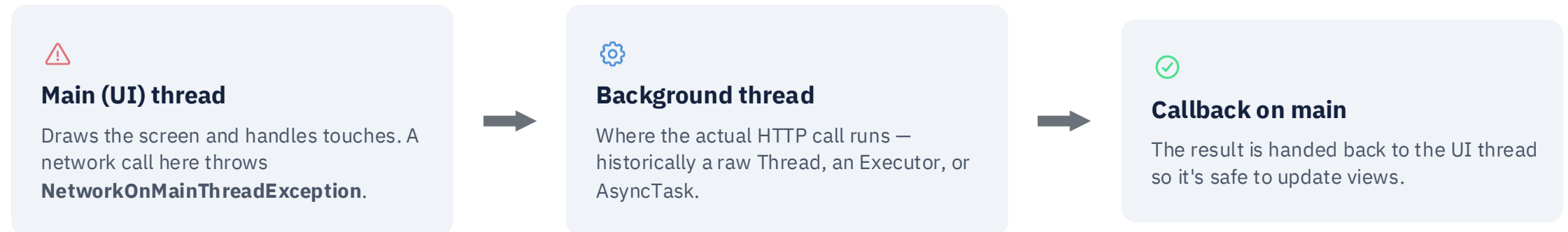
```
{  
  "id": 102,  
  "name": "Ananya Rao",  
  "department": "CSE",  
  "gpa": 8.7  
}
```

A server almost always answers in **JSON** — key-value pairs, arrays, and nested objects.

On Android, that text has to become a Kotlin/Java object (a **data class** or **POJO**) before your app can use it.

This mapping step is called **deserialization**, usually done by a library like **Gson** or **Moshi** — which is exactly what Retrofit and Volley plug in for you.

Basics: threads on Android



Both Volley and Retrofit manage this dance internally — you never touch a Thread directly.

Basics: manifest & setup

1. Permission

Every app that calls the internet needs this in **AndroidManifest.xml**, outside the **<application>** tag.

2. Dependency

Add the library to your module's **build.gradle** before you can use it.

```
<uses-permission  
    android:name=  
        "android.permission.INTERNET" />
```

```
implementation  
    "com.android.volley:volley:1.2.1"
```

```
implementation  
    "com.squareup.retrofit2:retrofit:2.11.0"
```

LIBRARY 1

Volley

Released by Google in 2013. Built for UIs that fire many small, fast requests — think a list that loads thumbnails as you scroll.

- Automatic request queuing & scheduling
- Built-in disk & memory response caching
- Request prioritization and cancellation
- Ready-made request types: String, JSON, Image



Best for: quick, frequent, lightweight calls

Library by Google

Volley

How to create a Request?

How about thread Management?

UI Thread

UI Thread

- Request (com.android.volley)
- ImageRequest (com.android.volley.toolbox)
- ClearCacheRequest (com.android.volley.toolbox)
- StringRequest (com.android.volley.toolbox)
- JsonRequest (com.android.volley.toolbox)
 - JsonObjectRequest (com.android.volley.toolbox)
 - JsonArrayRequest (com.android.volley.toolbox)

Worker Thread

Request added to Cache Queue

CacheDispatcher will dequeue the Request

Parse Response

Parsed Response on Main Thread

UI Thread

Cache thread

CacheDispatcher will dequeue the Request

Hit Backend, Parse

Network Threads

Write to Cache

How a Volley request flows



Volley in code

```
val queue = Volley.newRequestQueue(context)
val url = "https://api.jipmer.edu/students/102"

val request = JsonObjectRequest(
    Request.Method.GET, url, null,
    { response -> showStudent(response) },
    { error -> showError(error) }
)

queue.add(request)
```

Two callbacks — success and error — both delivered back on the main thread.

Retrofit

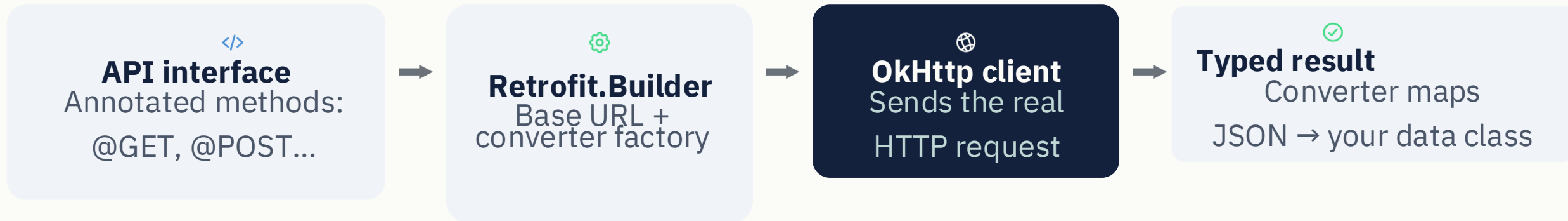
Built by Square. Turns a REST API into a plain Kotlin/Java **interface** — you declare endpoints, Retrofit generates the networking code.

- Type-safe endpoints via annotations
- Runs on OkHttp — connection pooling, interceptors
- Pluggable converters: Gson, Moshi, kotlinx.serialization
- First-class Kotlin coroutine & RxJava support



Best for: structured,
larger REST APIs

How Retrofit is wired together



Retrofit in code

```
interface ApiService {  
    @GET("students/{id}")  
    suspend fun getStudent(  
        @Path("id") id: Int  
    ): Student  
}
```

```
val retrofit = Retrofit.Builder()  
    .baseUrl("https://api.jipmer.edu/")  
    .addConverterFactory(  
        GsonConverterFactory.create()  
    ).build()  
  
val api = retrofit.create(ApiService::class.java)  
val student = api.getStudent(102)
```

The interface describes the API; the builder wires up the client — calling it reads like a normal function call.

Volley vs Retrofit

Aspect	Volley	Retrofit
Type safety	Manual, per request	Built in, via interface
JSON parsing	Basic, manual mapping	Automatic, via converters
Caching	Built-in disk/memory cache	Delegated to OkHttp
Best for	Small, frequent, simple calls	Large, structured REST APIs
Learning curve	Very shallow	Slightly steeper, pays off fast
Coroutine / Rx support	None built in	First-class
Maintained by	Google (community fork)	Square

Type safety & boilerplate

• Volley

```
val url = "https://api.jipmer.edu/students/102"

val request = JsonObjectRequest(
    Request.Method.GET, url, null,
    { response ->
        // untyped – nothing checks this
        // until it runs
        val name = response.getString("name")
        val age = response.getInt("age")
        showStudent(name, age)
    },
    { error -> showError(error) }
)
queue.add(request)
```

Volley: simple to start with, no interfaces or codegen — but the URL, field names, and types are only ever checked when the app runs.

• Retrofit

```
interface StudentApi {
    @GET("students/{id}")
    suspend fun getStudent(
        @Path("id") id: Int
    ): Student
}

// compiler-checked call, no manual parsing
val student = studentApi.getStudent(102)
showStudent(student.name, student.age)
```

Retrofit: the endpoint, parameters, and return type are all part of the interface — a typo or wrong type is caught by the **compiler**, not by a crash on a student's phone.

JSON parsing

• Volley

```
{ response ->
  // every field pulled out by hand
  val name = response.getString("name")
  val age  = response.getInt("age")
  val dept = response.getString("departement")
  // ^ typo'd key – compiles fine,
  //   crashes only when this line runs

  showStudent(Student(name, age, dept))
}
```

Volley: no dependency on a converter library — fine for one or two fields, but every new field means another manual `getString()/getInt()` line.

• Retrofit

```
data class Student(
  val name: String,
  val age: Int,
  val department: String
)

// Moshi/Gson converter maps JSON -> object
// automatically, using the field names above
val student: Student = studentApi.getStudent(102)
```

Retrofit: one data class describes the whole shape; a converter (Gson/Moshi) does the mapping — less code, and renaming a field is a one-line change, not a hunt through every request callback.

Caching

• Volley

```
// Volley owns its own disk cache
val cache = DiskBasedCache(cacheDir, 1024 * 1024)
val network = BasicNetwork(HurlStack())
val queue = RequestQueue(cache, network)
queue.start()

// every request is cache-checked
// automatically — no server cooperation needed
```

Volley: caching works even if the backend sends no cache headers at all — useful against APIs you don't control (handy for quick student projects hitting third-party APIs).

• Retrofit + OkHttp

```
val cache = Cache(cacheDir, 10L * 1024 * 1024)
val client = OkHttpClient.Builder()
    .cache(cache)
    .build()

val retrofit = Retrofit.Builder()
    .client(client)
    .baseUrl(BASE_URL)
    .build()

// caching follows the server's
// Cache-Control / ETag headers
```

Retrofit/OkHttp: standards-compliant HTTP caching (respects Cache-Control, ETag) — correct behavior for well-behaved APIs, but needs the server to send the right headers.

Which one should you reach for?



Choose Volley when

The app is small, requests are simple and frequent, or you're loading a lot of images in a list.



Choose Retrofit when

You're building against a real REST API, want type safety, and expect the app to grow.

Most modern Android apps default to Retrofit + OkHttp today — but understanding Volley's model is what makes Retrofit's design choices click.